

Relatório: Introdução e Hipóteses Informais

Arthur Lara Panzera
Felipe Augusto Pereira de Sousa
Gabriel Reis Lebron de Oliveira

19 de agosto de 2026

1 Introdução

Este trabalho tem como objetivo caracterizar repositórios populares open-source hospedados no GitHub, a partir da coleta de dados dos 1.000 repositórios com maior número de estrelas via API GraphQL do GitHub. A caracterização é guiada por seis questões de pesquisa, RQ01 a RQ06, cobrindo maturidade, contribuição externa, frequência de releases, frequência de atualização, linguagem de programação e percentual de issues fechadas dos repositórios, além da relação entre popularidade da linguagem e as demais métricas.

A seguir, apresentamos a hipótese informal levantada pelo grupo para cada RQ, formulada antes da análise formal dos dados, junto com uma validação preliminar da qualidade dos dados coletados (distribuição, valores ausentes e outliers).

2 Hipóteses Informais por Questão de Pesquisa

2.1 RQ01: Idade e maturidade dos repositórios

Pergunta: sistemas populares são maduros/antigos?

Métrica: idade do repositório (calculada a partir da data de criação).

Hipótese informal

Esperamos que repositórios populares sejam majoritariamente antigos, já que acumular um número alto de estrelas costuma exigir tempo de exposição e uso contínuo pela comunidade, tornando repositórios muito recentes uma minoria entre os mais populares.

Validação preliminar dos dados (1000 repositórios)

0% de valores ausentes. Mediana de 2829 dias (aproximadamente 7,7 anos), variando de 5 a 6703 dias (18,4 anos). Os cinco repositórios mais antigos (rails, git, jekyll, redis e jquery) foram criados entre 2008 e 2009, no início do GitHub, e seguem entre os mais populares, o que sustenta a hipótese. Ainda assim, há uma cauda de repositórios muito recentes, como o `deepseek-ai/deepseek-harness`, com apenas 5 dias de existência no momento da coleta, mostrando que projetos em áreas de grande interesse momentâneo, como IA, podem entrar no top 1000 rapidamente, contrariando o acúmulo gradual esperado.

2.2 RQ02: Volume de contribuição externa

Pergunta: sistemas populares recebem muita contribuição externa?

Métrica: total de pull requests aceitas.

Hipótese informal

Esperamos que repositórios populares recebam contribuição externa significativa, medida pelo número de pull requests mescladas, dado que maior visibilidade tende a atrair mais colaboradores dispostos a propor mudanças.

Validação preliminar dos dados (1000 repositórios)

0% de valores ausentes. Mediana de 768 pull requests mescladas, sustentando a hipótese de contribuição externa relevante. 20 repositórios (2%) aparecem com zero PRs mescladas, o que não significa ausência real de colaboração: `torvalds/linux` e `FFmpeg/FFmpeg` aceitam contribuições por lista de e-mail em vez do fluxo de pull request do GitHub. No extremo oposto, `firstcontributions/first-contributions` aparece com 103307 PRs mescladas, muito acima do segundo colocado; por ser um repositório-tutorial que aceita contribuições triviais deliberadamente, não é comparável aos demais projetos e deve ser tratado como outlier atípico nas análises agregadas.

2.3 RQ03: Frequência de releases

Pergunta: sistemas populares lançam releases com frequência?

Métrica: total de releases.

Hipótese informal

Esperamos que repositórios populares lancem releases com frequência regular, já que projetos com muita visibilidade tendem a ter ciclos de desenvolvimento mais estruturados e cadência de versionamento previsível.

Validação preliminar dos dados (1000 repositórios)

0% de valores ausentes. Mediana de 40 releases, máximo de 1000. 280 repositórios (28%) têm zero releases, a maioria listas de recursos ou material de referência que não usa o recurso de Releases do GitHub. 21 repositórios (2,1%) aparecem com exatamente 1000 releases, incluindo `langchain`, `next.js`, `llama.cpp`, `electron` e `storybook`. Confirmamos numa consulta separada à API REST que o `electron` tem 1981 releases reais, o que indica um teto na API do GitHub para esse campo, não um problema nos nossos dados.

2.4 RQ04: Frequência de atualização

Pergunta: sistemas populares são atualizados com frequência?

Métrica: tempo até a última atualização.

Hipótese informal

Esperamos que repositórios populares sejam atualizados com frequência alta, dado que atraem mais colaboradores e exigem manutenção constante para sustentar a base de usuários.

Validação preliminar dos dados (1000 repositórios)

0% de valores ausentes. Mediana de 2 dias desde a última atualização, com 33,1% dos repositórios atualizados no mesmo dia da coleta. 114 repositórios (11,4%) estão parados há mais de um ano, com os casos mais extremos passando de 6 anos sem atualização, em geral materiais de referência estáticos.

2.5 RQ05: Linguagem primária

Pergunta: sistemas populares são escritos nas linguagens mais populares?

Métrica: linguagem primária de cada repositório.

Pendência do grupo: definir e referenciar a fonte usada para “linguagens mais populares” (ex.: TIOBE Index, GitHub ou GitHub Octoverse), mantendo a mesma referência ao longo de todo o laboratório.

Hipótese informal

Esperamos que a maioria dos repositórios populares esteja concentrada nas linguagens de propósito geral mais usadas do mercado, ex.: JavaScript/TypeScript, Python, Java, já que essas linguagens têm o maior número de desenvolvedores e ecossistemas mais maduros para projetos open-source de grande escala.

Validação preliminar dos dados (1000 repositórios)

8,7% de valores ausentes, um total de 87 repositórios. 43 linguagens distintas identificadas. Top 3: Python 22,9%, TypeScript 17,4%, JavaScript 11,0%, 51,3% acumulado. Os valores ausentes não são erro de coleta: são repositórios sem código-fonte majoritário, cujo conteúdo é majoritariamente Markdown, então o GitHub não atribui linguagem primária de programação.

2.6 RQ06: Percentual de issues fechadas

Pergunta: sistemas populares possuem um alto percentual de issues fechadas?

Métrica: razão entre issues fechadas e total de issues.

Hipótese informal

Esperamos que repositórios populares tenham uma alta taxa de issues fechadas, já que a visibilidade atrai mantenedores e contribuidores que respondem e resolvem problemas relatados com mais agilidade.

Validação preliminar dos dados (1000 repositórios)

4,3% de valores ausentes, um total de 43 repositórios. Mediana de 87,51% de issues fechadas, mínimo de 7,69%, máximo de 100%. Os valores ausentes correspondem a repositórios

com zero issues registradas no total, alguns por serem listas sem rastreamento ativo, e um caso notável é o `linux`, que tem o rastreador de Issues desligado no GitHub porque o kernel usa lista de e-mails como canal oficial de bugs. Além disso, 26 repositórios (2,6%) têm razão de exatamente 100%. Confirmamos numa consulta separada que não é artefato de amostra pequena: o `nilbuild/developer-roadmap`, por exemplo, tem 3184 issues no total, todas fechadas, o que sugere bots de triagem automática ou política agressiva de fechamento.